Stop That Runner! — Development Document

Oct 24, 2026 · @Karthikeyan Kannan

Overview

Stop That Runner! is an endless reverse-runner for Android: the player controls the world and places traps, while an AI runner tries to escape. Yes, it is built as an endless game, and an on-device AI engine keeps creating new content by itself: new track layouts, new runner behaviors, new trap combinations and new challenges every session.

Item	Decision
Genre	Reverse runner (new genre): the player is the obstacle builder
Platform	Android first (phones, Android 8.0+), iOS later
Session length	2–10 minutes, endless until energy runs out
Audience	Casual and mid-core players, ages 12+
Engine	Unity (C#)
Business model	Free to play, cosmetics and optional rewarded ads

What “the AI creates new things itself” means in practice. The game does not need an internet AI to be endless. It combines four systems that run on the phone: a procedural world generator, a learning runner AI, a content combiner that invents new trap and runner variants from rules, and a difficulty director that keeps every run fresh. A cloud generative AI is an optional later add-on for new dialogue lines and themed events, not a requirement.

Story and setting

The player is ARC, an old city AI ordered by the city council to stop Kip, a runner who stole a glowing core. The story unfolds slowly inside the endless game, so it never interrupts play.

- **Setup:** Kip breaks into the city with a stolen core. The council tells ARC to stop him at any cost.

- **Rising doubt:** While running, Kip shouts lines like “You don’t understand!” Every 1,000 meters, a glitch file unlocks and reveals part of the city’s hidden past.
- **Twist:** The core is the city’s memory core. The council erased the truth, and Kip is trying to free the citizens.
- **Final choice:** In the last world, ARC either keeps trapping Kip (Order ending) or places traps that miss on purpose (Freedom ending).

How story fits an endless game. Story beats unlock by total distance across all runs, not in one run. After both endings, the game continues in endless mode with the AI engine generating new runners, so the story is a goal but never a limit.

Core gameplay

The player spends energy to place traps ahead of an auto-running AI; each crash scores points, and each checkpoint the runner passes drains energy. The run ends when energy reaches zero.

1. The runner starts automatically and the track scrolls endlessly.
2. The player places traps in the next 2–3 screens of track using energy.
3. Runner crashes: +score, +coins, energy refills 20%, and a new, slightly smarter runner spawns.
4. Runner reaches a checkpoint (every 300 m): energy −15%, runner speed +5%.
5. Energy at 0: run over, score saved to leaderboard.

Controls (one finger)

Gesture	Trap	Energy cost
Tap	Barrier wall	10
Swipe down	Hole	15
Swipe sideways	Rolling barrel	20
Hold 1 s	Big trap (falling crane)	40

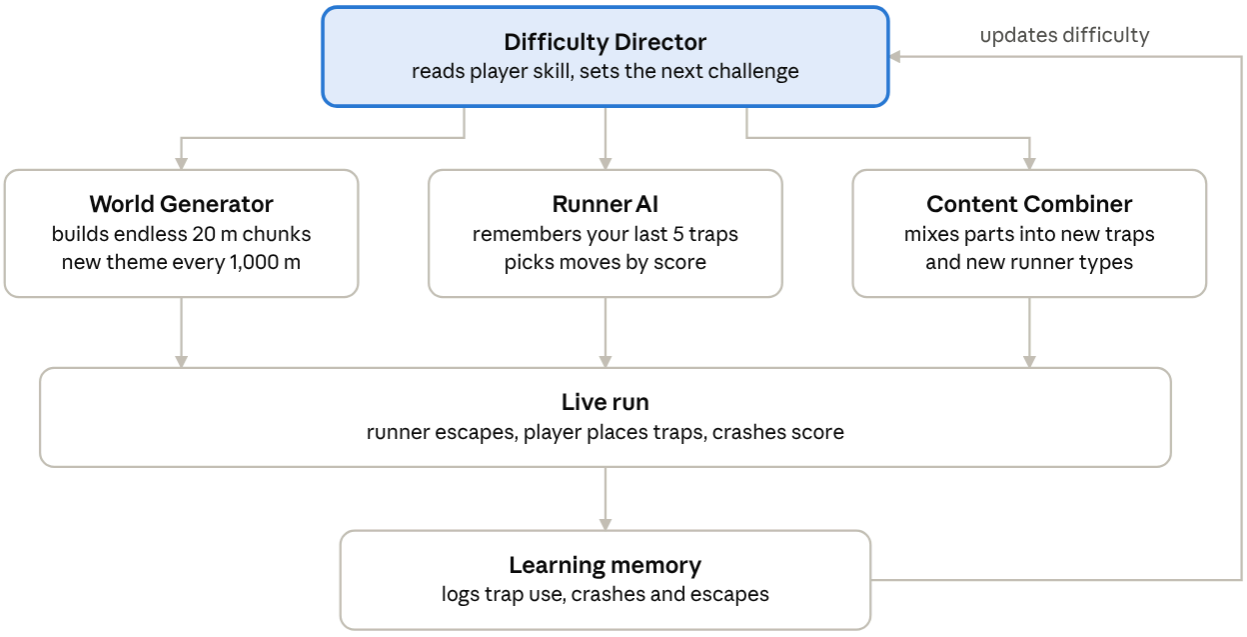
Scoring

- Base crash: 100 points.
- Combo (two traps chained within 2 s): ×1.5.
- Surprise bonus (a trap type the runner has not seen this run): +50.
- Repeat penalty: using the same trap 3 times in a row gives half points, which pushes creative play.

AI engine

The AI engine is five on-device systems in a loop: the Difficulty Director reads how well the player is doing and tells three generators what to build next, and every run feeds a memory that changes the next run.

Every run teaches the engine how to build the next one



AI engine · 5 systems in one loop

The loop means no two runs are the same, even after hundreds of hours.

1. World Generator (procedural track)

- Builds the track from 20 m chunks chosen by rules, such as “no two gaps in a row” and “leave room for at least 2 traps.”
- Every chunk is checked to be escapable, so the runner never dies without a trap.
- Changes theme every 1,000 m and blends themes during transitions.

2. Runner AI (learning opponent)

- Utility AI: each frame it scores options (jump, slide, switch lane, slow down) and picks the best.
- Learning memory: it tracks your last 5 trap types and raises the weight of the matching dodge, so a spammed trap stops working.
- Personality traits from the Content Combiner change the scores, so a cautious runner and a reckless runner behave differently.
- Optional upgrade: train the base behavior offline with Unity ML-Agents, then ship the trained model inside the app.

3. Content Combiner (creates new things)

- Traps = base + modifier + world element (for example, barrel + exploding + ice = freezing barrel).
- Runners = movement style + resistance + quirk + look (for example, jumper + wall-proof + fakes jumps + ninja skin).
- Every new combination is tested in a fast background simulation before it appears, so broken or impossible content is thrown away.
- 10 bases, 8 modifiers and 5 world elements give 400 trap variants; runner parts give thousands of runner types.

4. Difficulty Director

- Keeps a skill rating per player and targets a crash rate where the player wins about 60% of encounters.
- Raises speed, adds smarter runners or rarer variants when the player is winning easily; eases off after repeated escapes.

5. Learning memory

- Stores trap use, crashes, escapes and favorite combos, both for the current run and across runs.
- Feeds the Director and the Runner AI, and sends anonymous stats to analytics to tune balance.

Optional cloud generative AI (later phase). A language model can write new runner taunts, story side-chapters and event names, which are reviewed before release. The game must stay fully playable without it.

Worlds, modes and progression

The world changes theme every 1,000 meters inside one continuous run, and each theme adds its own trap and runner rules that the AI engine can mix.

World	Unique trap	Unique runner trait	Story unlock
City Streets	Traffic cone line	None (tutorial)	Kip’s first message
Jungle Temple	Swinging vines	Swings over holes	Council logs, part 1
Snow Mountain	Ice patch	Slides faster on ice	Erased citizen records
Space Station	Gravity flip	Floats briefly	Memory core blueprint
Candy Land	Sticky syrup	Bounces off walls	Final choice

Game modes

- **Endless:** the main mode, never ends while energy lasts.
- **Daily Generated Challenge:** the AI engine builds one seeded challenge per day, the same for every player worldwide.
- **Puzzle:** fixed trap set, one specific runner to stop.
- **Duel (later):** one player runs, the other builds traps, then they swap.

Progression

- Coins upgrade traps: lower energy cost, bigger area, faster setup.
- Player level unlocks new traps every 3 levels.
- Weekly leaderboard for most crashes in one run.
- Collection book of every runner type the AI has generated, which rewards seeing new ones.

Technical architecture

Everything that keeps the game endless runs offline on the phone; the backend only stores scores, the daily seed and saves.

Layer	Choice	Notes
Game engine	Unity 6 LTS, C#, URP 2D	Strong Android support, large asset store
Track generation	Chunk-based procedural system with object pooling	20 m chunks, max 6 loaded at once
Runner AI	Utility AI + lightweight learning memory	Optional: Unity ML-Agents for training base behaviors offline
Content generator	Rule-based combiner reading ScriptableObject data	Designers add parts; the generator mixes them
Randomness	Seeded RNG	Same seed = same daily challenge for everyone
Backend	Firebase or Unity Gaming Services	Auth, cloud save, leaderboards, remote config
Analytics	Firebase Analytics	Trap usage, run length, churn points
Ads / IAP	Unity LevelPlay or AdMob, Google Play Billing	Rewarded ads only

Performance targets

- 60 FPS on mid-range phones (4 GB RAM), 30 FPS minimum on low-end.
- Install size under 150 MB.
- AI decision time under 2 ms per frame.
- Fully playable offline.

Art, audio and UX

The look is bright 2.5D cartoon, so generated runner variants can be built from swappable body parts, colors and accessories.

- **Modular runner art:** head, body, legs, accessory and color layers; the content generator combines them so every new runner looks different.
- **Expressive reactions:** angry faces, dizzy stars and short voice lines when the runner crashes.
- **Crash replays:** 3-second slow-motion clip after great crashes, shareable as a short video.

- **Adaptive music:** layers add drums and bass as speed rises, and each world has its own theme.
- **UX:** energy bar and next-trap preview at the bottom, within thumb reach; first 60 seconds teach all four gestures with no text walls.
- **Accessibility:** colorblind-safe trap colors, haptics toggle, left-hand mode.

Monetization and live operations

The game is free with fair, cosmetic-first monetization; nothing sold makes traps stronger than earned upgrades.

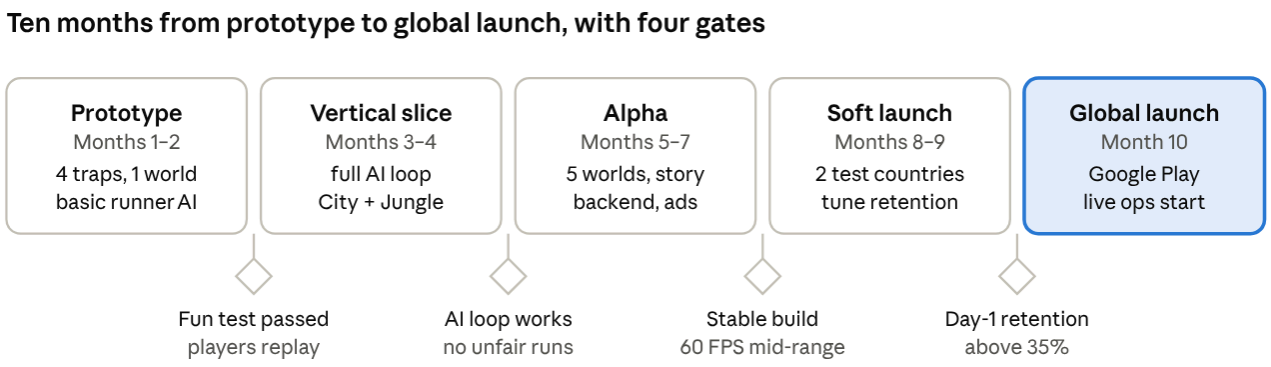
Revenue source	What players get	Price (USD)
Season pass	30 tiers of skins, trap effects, a themed world	4.99 per season
Cosmetic packs	Runner skins, trap skins, crash effects	0.99–2.99
Rewarded ads	Double coins or one extra energy refill per run	Free (optional)
Remove ads	Removes optional ad prompts	2.99 one-time

Live operations

- Weekly event: the AI engine generates a themed runner family (for example, “Ninja Week”) from existing parts plus one new part.
- Season every 8 weeks with a new world and story side-chapter.
- Remote config tunes energy costs and difficulty without an app update.

Roadmap, team, budget and risks

A small team of 5 can reach global launch in about 10 months; each phase must pass its gate before the next one starts.



Roadmap · 5 phases, 4 gates

If a gate fails, the team stays in that phase and fixes the problem rather than adding new content.

Team

Role	Count	Main work
Game designer / producer	1	Rules, balance, story, schedule
Gameplay developer (Unity)	1	Controls, traps, worlds, UI
AI and tools developer	1	Runner AI, generators, Director, simulation tests
2D artist / animator	1	Modular runner parts, worlds, effects
Sound designer (contract)	0.5	Adaptive music, voice lines
QA tester (part time)	0.5	Device testing, balance reports

Budget (approximate). Roughly 40,000–250,000 USD for 10 months, depending mostly on the team's country and salaries. A solo developer using asset-store art can build the prototype for under 1,000 USD in tools and assets.

Risks

Risk	Fallback
Runner AI feels unfair or too easy	Director caps AI learning speed; tune with playtest data
Generated content is broken	Background simulation rejects any variant that fails tests
Reverse role confuses new players	60-second guided tutorial; first runner is very slow
Low-end phones lag	Object pooling, fewer effects on low settings

Risk	Fallback
Players get bored after the story	Weekly generated events and seasons keep new content coming

First steps

- [ ] Build a one-screen Unity prototype: auto-runner + tap-to-place wall.
- [ ] Add the hole and barrel traps and a simple memory of the last 5 traps.
- [ ] Playtest with 10 people and check if they want to play again.